BizzConnect

Updated Full Project Plan & Technical Architecture

MANAGED COLLECTIVE PROCUREMENT PLATFORM

BUY • CONNECT • GROW

Technology baseline: MERN Stack
MongoDB • Express.js • React.js • Node.js

A procurement-first platform connecting small retailers with wholesalers through pooled demand, MOQ progress, supplier bidding, transparent allocations and a virtual fulfilment hub.

Planning baseline • Version 2.0 • 3 November 2026

1. Product vision and scope

BizzConnect is a managed B2B procurement platform connecting small retailers/kirana stores with wholesalers. Retailers pool demand to reach wholesale minimum order quantities (MOQs), unlock better price slabs, compare supplier offers and coordinate fulfilment through a virtual distribution hub.

The product makes collective purchasing transparent: pooled demand, threshold remaining, price tier, supplier terms, each retailer's allocation and the fulfilment journey are visible to the right user role.

Core pillars

- **Collective buying:** aggregate demand across independent retailers so each can access wholesale economics without buying the entire MOQ alone.
- **Intelligent procurement:** begin with explainable rules for grouping demand, landed-price comparison, reminders, threshold recovery and backup suppliers.
- **Collective negotiation:** invite supplier bids/counter-offers against a defined pooled requirement and compare price, delivery, MOQ and reliability.

MVP boundaries

- Simulate payment and logistics; do not collect real money or trigger live shipping integrations in the initial MVP.
- Use a transparent transaction commission as the primary revenue model.
- Build realistic order, bid, allocation, hub, delivery and dispute states even when money movement and logistics are simulated.
- Keep advanced forecasting, external supplier integrations and sophisticated AI decisioning as later enhancements.

User roles

Role	Goals and workspace
Retailer	Discover products; join group orders; compare bids/deals; track orders; see receipts, savings and inventory.
Wholesaler	Manage catalogue, MOQ and price slabs; respond to bids; accept purchase orders; update fulfilment.
Admin / Operations	Verify suppliers; oversee groups, bids, payment simulation, hub, delivery, disputes and platform metrics.

2. Product and visual experience direction

The website must feel like a purpose-built procurement operations product rather than a generic dashboard. Quantities, rates, MOQs, supplier terms, allocations and fulfilment states should drive the information hierarchy.

Confirmed design system

Decision	Direction
Theme	Modern Trade; premium; balanced information density.
Palette	Alpine Teal: #123F42 deep teal; #247A76 muted turquoise; #D5A46C warm copper; #F3F0E8 soft ivory; #252D2C deep charcoal.
Modes	Light and dark themes with semantic surface, text, border, focus and status tokens.
Logo	Connected geometric B mark + BizzConnect wordmark; scalable SVG variants.
Navigation	Expandable sidebar; role-appropriate navigation; persistent topbar.
Metrics	Number ticker for meaningful values such as savings, pooled quantity and procurement spend.
Charts	Only where they answer an operational question; no decorative dashboard charts.

Accessibility and motion

- Use copper as an accent, not small body text or a success colour. Define success/warning/danger/info tokens separately.
- Verify contrast in both themes and provide visible keyboard focus.
- First load: connected-B paths assemble and wordmark fades in; target about 1.5 seconds without delaying usable content.
- Slow network: content-shaped skeletons with shimmer; no full-screen logo loader.
- Navigation: fade and slight upward motion around 200 ms. Animate page content, not persistent sidebar/topbar.
- Use subtle card lift, animate MOQ progress and meaningful number changes, include loading/empty/error/retry states and respect reduced-motion preferences.

3. Recommended MERN technical architecture

Use separate frontend and backend applications, ideally in a monorepo with independently runnable apps and shared API contracts. Build the React frontend first against realistic mock data while Express API contracts and MongoDB document models are stabilised.

Required technology stack

Layer	Technology	How it is used in BizzConnect
Frontend	React.js + TypeScript + Vite	Builds the retailer, wholesaler and admin interfaces using reusable components, hooks, routing and API-driven state.
Backend runtime	Node.js + TypeScript	Runs the server-side application and domain services.
API framework	Express.js	Defines REST routes, middleware, authentication, validation, controllers and centralized error handling.
Database	MongoDB	Stores users, businesses, catalogues, demand groups, bids, orders, allocations, fulfilment events and audit records as documents/collections.
ODM	Mongoose	Defines schemas, field validation, references, indexes, model methods and database access.
UI / styling	Tailwind CSS + shadcn/ui + Lucide React	Accessible primitives customised to the BizzConnect identity.
Motion / charts	Motion for React; Recharts	Purposeful transitions and operational analytics.
Validation	Zod or express-validator	Validates incoming API payloads; shared safe DTO schemas where practical.
Testing	Vitest, React Testing Library, Supertest, Playwright	Unit, component, API integration and end-to-end user journey tests.

System flow

Browser → React application → typed API client (fetch/Axios) → Express REST API → middleware (authentication, authorization, validation) → route/controller → service/business rules → Mongoose models → MongoDB.

React owns presentation and interaction state. Express owns the API boundary and request lifecycle. Service modules own procurement rules. Mongoose models define and validate persisted documents. MongoDB is the persistent source of truth.

Suggested repository structure

```
bizzconnect/
■■■ apps/
■ ■■■ web/ # React + TypeScript + Vite
■ ■ ■■■ src/{app,assets,components,features,hooks,lib,styles,types}/
■ ■■■ api/ # Node.js + Express + TypeScript
■ ■■■ src/{config,middleware,modules,services,models,routes,utils}/
■■■ packages/
■ ■■■ contracts/ # shared DTOs, schemas, enums
■ ■■■ config/ # shared TS/lint configuration
■■■ docs/ # requirements, architecture, API, decisions
■■■ progress.md # mandatory agent work log
■■■ .env.example
■■■ README.md
```

Architecture rule: do not put procurement calculations in React components or Express route handlers. React components display state; Express handlers validate and delegate; domain services own business rules; Mongoose models/repositories own persistence.

4. Frontend plan — React phases and parts

Build the React frontend first using mock data and stable API contracts. Every important screen must include loading, empty, error, success and permission-denied states.

F0 — React foundation

- Create React + TypeScript + Vite app, scripts, lint/format, environment handling and folder structure.
- Configure React Router for public, retailer, wholesaler and admin routes; add protected route layouts.
- Define Alpine Teal tokens: light/dark themes, typography, spacing, radii, shadows, borders and status colours.
- Convert approved logo to SVG variants: wordmark, standalone mark, light/dark, favicon and app icon.
- Build app shell: expandable sidebar, topbar, breadcrumbs, profile menu and responsive navigation.
- Implement logo assembly, skeleton shimmer, route transitions, reduced-motion support and toast/notification patterns.

F1 — Shared React interaction system

- Create reusable React components: buttons, inputs, selects, dialogs, dropdowns, tabs, tables, badges, cards, tooltips and pagination.
- Create domain components: price display, quantity stepper, MOQ meter, price ladder, supplier reliability, order status and savings display.
- Use custom hooks for authentication, current user, pagination, filters and API requests.
- Create a typed API client layer; keep request logic out of presentational components.
- Add responsive breakpoints, keyboard navigation, focus visibility and accessible labels.

F2 — Public and account experience

- Landing page explaining pooled demand, bulk pricing, trust and the managed process.
- Sign-in, registration, password recovery and onboarding; production authentication comes from Express backend.
- Role-specific retailer/wholesaler onboarding and business profile.

F3 — Retailer workspace

- Overview: active groups, savings, procurement spend, pending actions and recent activity.
- Marketplace: category/search filters, pack sizes, MOQ, price slabs, supplier and availability.
- Product detail: quantity selector, price ladder, landed-price breakdown and join/create group action.
- Collective order: pooled quantity, MOQ, participant contributions, deadline, milestones and remaining amount.
- Bids/deals: compare unit price, landed price, delivery, MOQ and reliability.
- Checkout simulation: retailer allocation, fee/commission disclosure, simulated payment and receipt.
- Tracking: supplier confirmed → hub received → sorting → dispatch → delivery; timestamps and exceptions.
- Inventory, reorder reminders, price/savings history and Retailer Power Index.

F4 — Wholesaler workspace

- Dashboard: bid invitations, open pooled demand, accepted POs, fulfilment queue and performance.
- Catalogue management: products, pack sizes, availability, MOQ and quantity price slabs.
- Bid inbox: requirement, quantity, deadline, submit/revise bid, delivery terms and notes.
- Accepted purchase orders, fulfilment milestones, hub handover and issue reporting.

4. Frontend plan — continued

F5 — Admin and operations workspace

- Operations overview: groups near deadline, at-risk orders, verification queue, hub tasks and disputes.
- Supplier verification and catalogue moderation.
- Group controls: threshold monitoring, close/reopen policy, exceptions and backup supplier workflow.
- Bid comparison and award oversight with audit trail.
- Virtual hub receiving, sorting, allocation, dispatch and delivery simulation.
- Disputes, evidence, adjustments simulation, support notes and resolution history.
- Metrics: pooled volume, conversion, fulfilment reliability, savings, commission and supplier performance.

F6 — React quality and handoff

- Replace mock data feature-by-feature with calls through the typed API client to Express endpoints.
- Add route guards for user experience; rely on Express for real authorization.
- Playwright tests for critical journeys across all three roles.
- Check mobile/tablet layouts, accessibility, loading/error/offline states and production build.

5. Backend plan — Express.js + MongoDB phases

The backend must be independently runnable, secure by default and authoritative for every calculation and state change. Express.js exposes the REST API; Mongoose communicates with MongoDB.

B0 — Express and MongoDB foundation

- Create Node.js + TypeScript + Express app, environment config, health endpoint, structured logs and centralized error middleware.
- Connect to MongoDB using Mongoose; configure connection lifecycle, environment-specific database URLs and graceful shutdown.
- Create initial Mongoose models, schema validation, indexes, seed data and test database configuration.
- Add API versioning, request validation, response envelopes, pagination and machine-readable error codes.
- Implement authentication, password hashing, session/token lifecycle, logout, recovery and rate limits.
- Implement server-side role and resource-level access for retailer, wholesaler and admin.
- Configure security headers, CORS allowlist, CSRF protection where cookie auth is used, input validation and safe uploads.
- Add audit logs for admin actions, supplier decisions, bid awards, order adjustments and dispute resolution.

B1 — Identity and suppliers

- Create User, RetailerProfile, WholesalerProfile, BusinessAddress and SupplierVerification collections.
- Implement registration, login, profile management, supplier onboarding and verification review endpoints.
- Store password hashes only; never store plaintext passwords.
- Add permission tests to prove users cannot access another business's private records.

B2 — Catalogue and pricing engine

- Create Category, Product, SKU, SupplierListing, PriceSlab and InventorySnapshot collections.
- Support products, pack sizes, units, supplier listings, availability, MOQ and quantity price slabs with effective dates.
- Implement landed-price calculation: unit price plus explicitly modelled delivery, fees and commission.
- Test slab boundaries, rounding, unit conversion and invalid configurations.

B3 — Collective demand and MOQ engine

- Create BuyingClub, ClubMembership, DemandGroup, DemandContribution and GroupMilestone documents.
- Implement retailer contribution/reservation records; validate quantity and eligibility.
- Calculate threshold, remaining quantity/value, milestones and deadline handling on the server.
- Support reminders, more participants, policy-based extension or backup supplier when a threshold is at risk.
- Prevent over-allocation, duplicate joins and race conditions using atomic updates, unique indexes and MongoDB transactions where multi-document consistency is required.

5. Backend plan — continued

B4 — Reverse auction and negotiation

- Create BidInvitation, SupplierBid, BidRevision and BidAward collections.
- Implement supplier invitation and bid-window lifecycle; enforce submission, revision and withdrawal rules.
- Maintain bid history and compare unit/landed price, MOQ, delivery, availability and reliability.
- Implement admin award workflow with recorded rationale and audit trail.
- Calculate supplier reliability from documented inputs; do not label an unexplained score as AI.

B5 — Orders, allocation and simulated checkout

- Create PurchaseOrder, RetailerOrder, Allocation, OrderLine, Receipt and CommissionRecord documents.
- Create purchase orders from awarded bids and accepted group demand.
- Allocate quantity and cost shares to retailers; define short-supply and rounding-remainder rules.
- Generate retailer order summaries, receipts and commission disclosure.
- Simulate pending/paid/failed/refunded states without a real payment gateway.
- Use idempotency keys and unique constraints for checkout/order creation to prevent duplicate orders.

B6 — Virtual hub, delivery and exceptions

- Create Hub, HubReceipt, SortingBatch, DispatchBatch, Delivery and DeliveryEvent collections.
- Support hub receipt, quantity checks, discrepancies and receiving evidence metadata.
- Support sorting, retailer allocation, dispatch batches and delivery status.
- Record delivery milestones and proof-of-delivery simulation.
- Handle delay, short shipment, damaged goods, supplier cancellation and failed delivery.
- Manage disputes, evidence, assignment, resolution, adjustment simulation and audit trail.

B7 — Analytics and automation

- Store historical prices, savings and retailer procurement summaries.
- Calculate Retailer Power Index with transparent inputs and explanation.
- Generate reorder reminders, expiring group alerts and threshold recovery notifications.
- Provide rule-based backup supplier suggestions and exception alerts.
- Expose admin metrics: pooled volume, fill rate, on-time delivery, supplier response, disputes and commission.

B8 — Express backend quality and release

- Unit tests for domain rules; Supertest integration tests for Express routes and MongoDB boundaries.
- Authorization, abuse, validation, rate-limit and concurrency tests.
- OpenAPI docs, example requests/responses, repeatable seed/demo data and local setup.
- Backup/restore plan, health checks, logs, metrics and deployment configuration.

MongoDB implementation expectations

MongoDB capability	BizzConnect use
Collections and documents	Persist users, products, supplier listings, groups, bids, orders, fulfilment events and audit logs.
Mongoose schemas	Define required fields, types, enums, defaults, validation and references for each collection.
Indexes	Index email, SKU, supplierId, group status/deadline, orderId and commonly filtered fields; use unique indexes where required.
References and population	Reference related documents such as user, supplier, demand group and purchase order; populate only where useful.

MongoDB capability	BizzConnect use
Atomic updates	Safely increment pooled quantities and update counters without lost updates.
Transactions	Use MongoDB transactions for critical multi-document workflows when the deployment runs on a replica set.
Aggregation pipeline	Compute operational summaries, pooled totals, supplier comparisons and analytics.

6. MongoDB data model and collections

MongoDB stores domain records as BSON documents grouped into collections. Use Mongoose models to define document shape, validation, indexes and relationships. Embed small data that is normally read and updated together; reference independently managed or potentially large related records.

Domain	MongoDB collections / Mongoose models	Purpose
Identity	users, retailerProfiles, wholesalerProfiles, supplierVerifications	Accounts, roles, business profiles and supplier trust.
Catalogue	categories, products, skus, supplierListings, priceSlabs, inventorySnapshots	Products, pack/unit, availability and price tiers.
Collective demand	buyingClubs, clubMemberships, demandGroups, demandContributions, groupMilestones	Pooled demand, MOQ progress and retailer contributions.
Negotiation	bidInvitations, supplierBids, bidRevisions, bidAwards	Supplier bids, revisions, comparisons and awards.
Orders	purchaseOrders, retailerOrders, allocations, receipts, commissionRecords	POs, retailer shares, allocation and transaction summary.
Payment simulation	paymentSimulations, paymentEvents, adjustments	Non-real payment states and adjustment history.
Fulfilment	hubs, hubReceipts, sortingBatches, dispatchBatches, deliveries, deliveryEvents	Virtual hub and delivery tracking.
Support / governance	disputes, disputeEvidence, disputeResolutions, auditLogs, notifications, priceHistory	Exceptions, traceability, alerts and analytics.

Document design rules

- Use stable ObjectId identifiers and explicit createdAt/updatedAt timestamps.
- Use references for independently managed entities and large/high-growth histories; avoid unbounded arrays inside a single document.
- Embed compact snapshots when historical records must preserve the price, product name or supplier terms that applied at order time.
- Define unique indexes for user email, SKU within its intended scope, and idempotency keys.
- Add compound indexes that match real query patterns, for example status + deadline or supplierId + createdAt.
- Use schema validation at the API boundary and Mongoose schema level; never rely on frontend validation alone.

State-machine rule

Demand groups, bids, purchase orders, simulated payments, fulfilment and disputes need explicit allowed state transitions. The browser must never be trusted to jump an order from draft to delivered or award a bid without Express-side validation and authorization.

Consistency and concurrency note

MongoDB provides atomicity at the single-document level. For workflows that update multiple documents together—such as awarding a bid and creating linked purchase orders—use transactions on a replica-set deployment, with clear retry handling. Also use conditional updates, unique indexes and idempotency keys to protect against duplicate or concurrent requests.

7. Express REST API contract strategy

Use REST endpoints under /api/v1, implemented with Express routers and controllers. Document them in OpenAPI and share DTO/schema definitions where practical. Do not expose raw Mongoose documents directly; map them to response DTOs.

Module	Example endpoint family	Operations
Auth / users	/api/v1/auth, /api/v1/me	Register, login, logout, session/refresh, profile.
Suppliers	/api/v1/suppliers	Onboard, verify, list, update status.
Catalogue	/api/v1/products, /api/v1/supplier-listings	Search, details, listings, price slabs.
Clubs / demand	/api/v1/clubs, /api/v1/demand-groups	Join, contribute, view progress, close.
Bids	/api/v1/bid-invitations, /api/v1/bids	Invite, submit/revise, compare, award.
Orders	/api/v1/purchase-orders, /api/v1/retailer-orders	Create, allocate, view, confirm.
Fulfilment	/api/v1/hub, /api/v1/deliveries	Receive, sort, dispatch, track.
Disputes / analytics	/api/v1/disputes, /api/v1/analytics	Resolve issues; savings and operations metrics.

Express request lifecycle

Incoming request → CORS/security middleware → authentication middleware → role/resource authorization → validation middleware → route/controller → service → Mongoose model → MongoDB → response DTO.

Example Express module structure

```
modules/
■■■ auth/{auth.routes.ts,auth.controller.ts,auth.service.ts}
■■■ products/{product.routes.ts,product.controller.ts,product.service.ts}
■■■ demand-groups/{group.routes.ts,group.controller.ts,group.service.ts}
■■■ bids/{bid.routes.ts,bid.controller.ts,bid.service.ts}
■■■ orders/{order.routes.ts,order.controller.ts,order.service.ts}
models/ # Mongoose schemas and models
middleware/ # auth, roles, validation, error handler
config/ # environment and MongoDB connection
```

API conventions

- Validate every request at the Express API boundary; return field-level validation errors.
- Use pagination, explicit sorting/filtering and stable identifiers.
- Use a consistent error shape with machine-readable code and safe message.
- Use idempotency keys for checkout and non-repeatable operations.
- Never trust browser-supplied prices, totals, roles, allocations or status changes; recompute on the server.
- Keep controllers thin; move business logic into services and database operations into model/repository functions.

8. Security and quality gates

- Use modern password hashing; never store plaintext passwords.
- Enforce server-side role and resource authorization on every protected Express operation.
- Apply strict input validation, safe output handling, parameterized/query-safe database access and upload restrictions.
- Use HTTPS in deployed environments and secure cookie settings for cookie-based sessions.
- Configure CORS allowlist, security headers, CSRF protection where applicable, rate limits and brute-force protections.
- Keep secrets out of Git; use .env.example with placeholders.
- Audit supplier verification, bid awards, admin actions and order/dispute adjustments.
- Do not expose stack traces, secrets or unnecessary personal data in errors/logs.
- Use atomic updates, unique indexes, idempotency and MongoDB transactions for critical concurrent workflows.
- Back up MongoDB and test restore procedures before production.

Quality gates

Gate	Evidence required
React frontend feature	Type-check, lint, component/feature tests, responsive and keyboard checks.
Express backend feature	Domain unit tests, Supertest API integration tests, authorization tests.
MongoDB data layer	Mongoose schema validation, index checks, seed verification, transaction/concurrency tests where relevant.
Shared contract	Schema validation and request/response examples verified by frontend and backend.
Workflow	Playwright end-to-end test for complete journey and failure path.
Release	Production build, security review, environment review, docs and no unresolved critical defects.

Environment variables (example names)

```
# API
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://127.0.0.1:27017/bizzconnect
JWT_ACCESS_SECRET=replace_me
JWT_REFRESH_SECRET=replace_me
CLIENT_ORIGIN=http://localhost:5173

# React app (.env for Vite)
VITE_API_BASE_URL=http://localhost:5000/api/v1
```

Never commit real secrets. For production, use a managed secret store and a MongoDB deployment configured with authentication, network restrictions, backups and replica-set support if transactions are required.

9. Avoiding generic AI-generated design

- Show real procurement facts: SKU, pack size, unit, current price, next price tier, MOQ, pooled quantity, deadline, landed cost and supplier terms.
- Build signature components: Collective Order Progress Card, Supplier Bidding Arena, Bulk Price Ladder, Procurement Journey Tracker, Savings Intelligence Panel and Retailer Power Index.
- Compare suppliers by landed price, MOQ fit, delivery, availability and reliability—not just decorative star ratings.
- Use believable sample products, quantities, prices and operational statuses; label demo data clearly.
- Give retailer, wholesaler and operations users distinct dashboards and priorities.
- Use strong typography, deliberate whitespace, readable tables, useful filters and meaningful empty states.
- Keep animation purposeful; avoid constant floating blobs, excessive glassmorphism, gradients and glow.

10. Build order and dependencies

Step	Work	Dependency
1	Confirm product rules, role permissions, MVP exclusions and core states.	Product decisions.
2	Create monorepo, conventions, progress.md, design tokens, MongoDB data plan and API contracts.	Step 1.
3	Build React app shell, logo assets, themes, navigation and loading patterns.	Step 2.
4	Build retailer flows in React using realistic mock data.	Step 3; domain rules.
5	Build wholesaler and admin React workflows using shared components.	Step 3.
6	Implement Express foundation, MongoDB connection, Mongoose models, identity and security.	Domain model/contracts.
7	Implement catalogue, pricing, demand/MOQ and bidding services/endpoints.	Backend foundation.
8	Implement orders, allocation, payment simulation and fulfilment.	Demand and bidding.
9	Integrate React feature-by-feature with Express APIs and run end-to-end journeys.	Stable API modules.
10	Hardening, accessibility, performance, docs and release.	Integrated product.

11. MVP completion criteria

- Retailer can discover a product, understand MOQ/price ladder, join a collective order, see pooled progress and receive an allocation/receipt.
- Wholesaler can publish listings/price slabs, respond to a bid invitation, receive an awarded PO and update fulfilment.
- Admin can verify suppliers, monitor group orders, oversee awards, manage hub stages and resolve disputes.
- React interfaces support all three roles and consume real Express REST endpoints for implemented features.
- Express validates requests, authenticates users, enforces permissions and applies business rules server-side.
- MongoDB persists users, products, demand groups, bids, orders, allocations, fulfilment and audit records through Mongoose models.
- MOQ, price tier and allocation are computed server-side and tested at boundary cases.
- Payment and logistics are explicitly simulated; no live financial transaction or shipment is initiated.
- Critical actions are auditable; frontend has coherent themes, responsive controls, skeletons, errors/retry and reduced motion.

12. Agent protocol and progress.md

The root-level progress.md is a mandatory work journal. It must capture intended changes before implementation, then actual changes and verification evidence afterward.

Required implementation loop

Step	Required action
1. Inspect	Inspect relevant React components, Express routes/services, Mongoose models, tests, dependencies and current conventions.
2. Plan first	Add task ID, workstream, reason, files to inspect/change, exact planned changes, risks and verification plan to progress.md.
3. Save	Write progress.md to disk before editing implementation files.
4. Implement	Stay within the plan; update the log before expanding scope.
5. Verify	Run relevant unit/integration/E2E tests, lint, type-check, build and manual checks.
6. Record	Only after checks pass, mark SUCCESS and include actual command/result evidence. Otherwise mark FAILED or BLOCKED.
7. Close	Update phase status, known issues, blockers and next task.

Required task record example

```
### [BE-014] — Implement demand group contribution endpoint
Status: PLANNED
Phase: B3 | Workstream: BACKEND (Express + MongoDB)
Why needed: Persist retailer contributions and maintain correct MOQ progress.
Files to inspect: demand-group routes, service, Mongoose models, tests
Expected files to change: group.routes.ts, group.controller.ts, group.service.ts, DemandGroup.model.ts
Planned changes: validate contribution, authorize retailer, apply safe atomic update and return DTO
Risks/dependencies: concurrent joins; unique membership/contribution constraints
Verification plan: service unit test + Supertest integration test + concurrency test
Actual changes: (fill after implementation)
Verification evidence: (commands and actual results)
Outcome / next task: ...
```

Hard rule: writing code is not proof of success. Never fabricate output, erase history or mark a task successful before its verification plan passes.

13. Business rules to confirm before backend implementation

- Are buying groups open neighbourhood pools, invite-only clubs, or both?
- Can a retailer leave or reduce quantity after joining? What is the cutoff?
- If MOQ is not reached: cancel, extend under policy, or switch supplier?
- Who selects the winning bid: best-value rule, admin approval, or retailer vote?
- How are delivery, hub handling, commission and other fees allocated?
- What happens with short supply, damaged goods or supplier cancellation?
- What unit, currency and rounding rules apply to pack sizes and retailer allocations?

Recommended starting point

Begin at Phase 0: confirm the business rules, create the repository and progress.md, establish the React app shell, Express API skeleton, MongoDB connection and Mongoose model conventions, then define shared API contracts. Build role-based React experiences with mock data before connecting backend modules.

Technology usage summary for evaluation

Technology	Required implementation evidence
React.js	Component-based UI, React Router role workspaces, hooks/state, reusable forms/tables, API integration and responsive screens.
Express.js	REST API routes, controllers, middleware, authentication/authorization, validation, centralized errors and service delegation.

Technology	Required implementation evidence
MongoDB	Database connection, collections/documents, Mongoose schemas/models, CRUD operations, indexes, references and persistence of core workflows.